

## UNIT-3

RMI (Remote Method Invocation) RMI overview RMI architecture, Designing RMI application, Executing RMI application. Example demonstrating RMI

## RMI (Remote Method Invocation)

The **RMI** (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java.

The RMI allows an object to invoke methods on an object running in another JVM.

The RMI provides remote communication between the applications using two objects **stub** and **skeleton**.

### Understanding stub and skeleton

RMI uses stub and skeleton object for communication with the remote object.

A **remote object** is an object whose method can be invoked from another JVM.

### stub

The stub is an object, acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes method on the stub object, it does the following tasks:

1. It initiates a connection with remote Virtual Machine (JVM),
2. It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
3. It waits for the result
4. It reads (unmarshals) the return value or exception, and
5. It finally, returns the value to the caller.

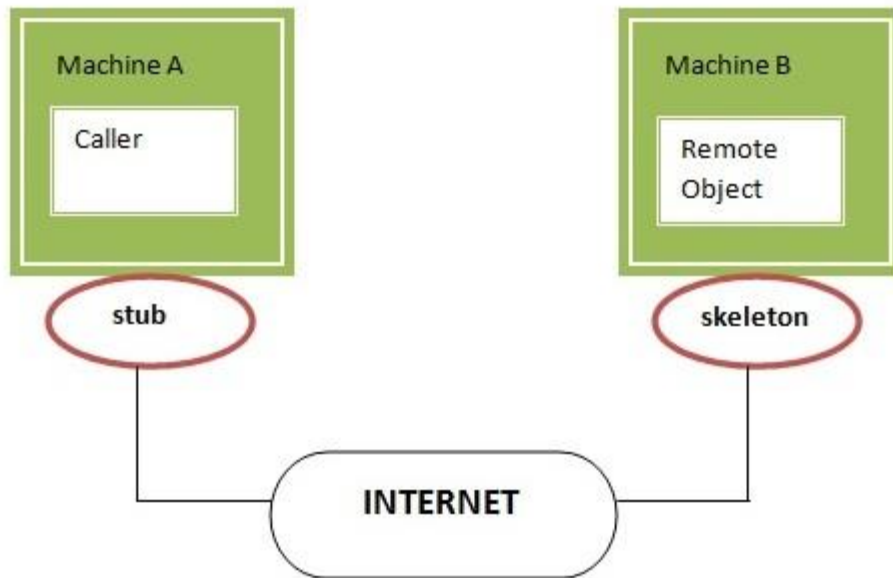
### skeleton

The skeleton is an object, acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:

1. It reads the parameter for the remote method
2. It invokes the method on the actual remote object, and

3. It writes and transmits (marshals) the result to the caller.

In the Java 2 SDK, an stub protocol was introduced that eliminates the need for skeletons.



## Understanding requirements for the distributed applications

If any application performs these tasks, it can be distributed application.

.

1. The application need to locate the remote method
2. It need to provide the communication with the remote objects, and
3. The application need to load the class definitions for the objects.

The RMI application have all these features, so it is called the distributed application.

---

## Java RMI Example

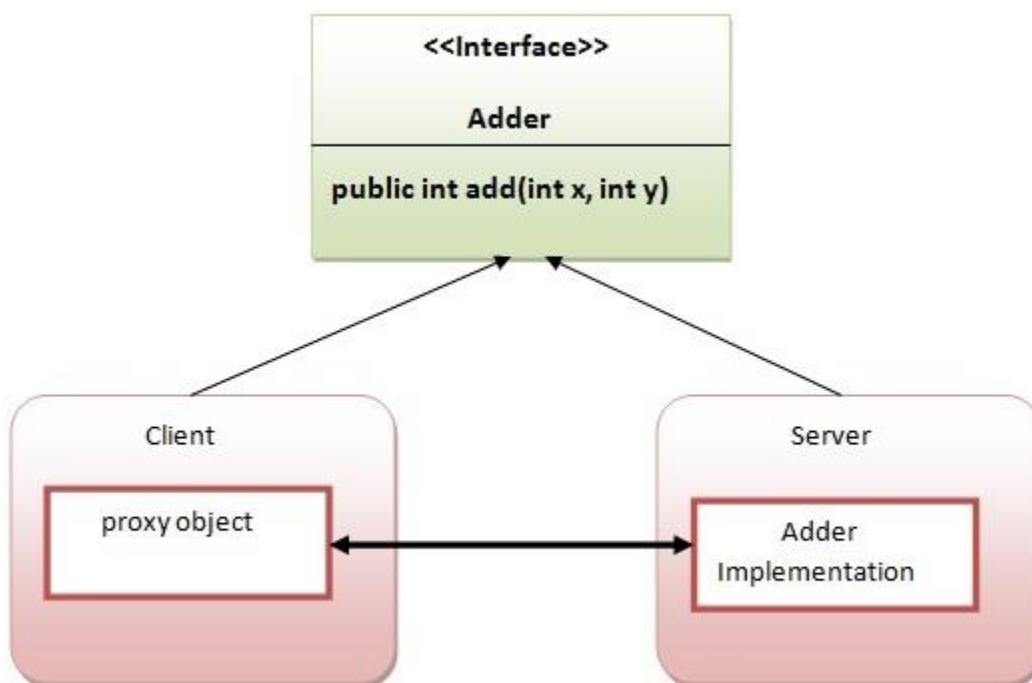
The is given the 6 steps to write the RMI program.

1. Create the remote interface

2. Provide the implementation of the remote interface
3. Compile the implementation class and create the stub and skeleton objects using the `rmic` tool
4. Start the registry service by `rmiregistry` tool
5. Create and start the remote application
6. Create and start the client application

## RMI Example

In this example, we have followed all the 6 steps to create and run the rmi application. The client application need only two files, remote interface and client application. In the rmi application, both client and server interacts with the remote interface. The client application invokes methods on the proxy object, RMI sends the request to the remote JVM. The return value is sent back to the proxy object and then to the client application.



### 1) create the remote interface

For creating the remote interface, extend the `Remote` interface and declare the `RemoteException` with all the methods of the remote interface. Here, we are creating a remote interface that extends the `Remote` interface. There is only one method named `add()` and it declares `RemoteException`.

1. **import** java.rmi.\*;
  2. **public interface** Adder **extends** Remote{
  3. **public int** add(**int** x,**int** y)**throws** RemoteException;
  4. }
- 

## 2) Provide the implementation of the remote interface

Now provide the implementation of the remote interface. For providing the implementation of the Remote interface, we need to

- Either extend the UnicastRemoteObject class,
- or use the exportObject() method of the UnicastRemoteObject class

In case, you extend the UnicastRemoteObject class, you must define a constructor that declares RemoteException.

1. **import** java.rmi.\*;
  2. **import** java.rmi.server.\*;
  3. **public class** AdderRemote **extends** UnicastRemoteObject **implements** Adder{
  4. AdderRemote()**throws** RemoteException{
  5. **super**();
  6. }
  7. **public int** add(**int** x,**int** y){**return** x+y;}
  8. }
- 

## 3) create the stub and skeleton objects using the rmic tool.

Next step is to create stub and skeleton objects using the rmi compiler. The rmic tool invokes the RMI compiler and creates stub and skeleton objects.

1. rmic AdderRemote
- 

## 4) Start the registry service by the rmiregistry tool

Now start the registry service by using the rmiregistry tool. If you don't specify the port number, it uses a default port number. In this example, we are using the port number 5000.

1. rmiregistry 5000
-

## 5) Create and run the server application

Now rmi services need to be hosted in a server process. The Naming class provides methods to get and store the remote object. The Naming class provides 5 methods.

|                                                                                                                                                                          |                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>public static java.rmi.Remote lookup(java.lang.String) throws java.rmi.NotBoundException, java.rmi.RemoteException, java.net.MalformedURLException;</code>         | It returns the reference of the remote object.                                |
| <code>public static void bind(java.lang.String, java.rmi.Remote) throws java.rmi.AlreadyBoundException, java.rmi.RemoteException, java.net.MalformedURLException;</code> | It binds the remote object with the given name.                               |
| <code>public static void unbind(java.lang.String) throws java.rmi.RemoteException, java.rmi.NotBoundException, java.net.MalformedURLException;</code>                    | It destroys the remote object which is bound with the given name.             |
| <code>public static void rebind(java.lang.String, java.rmi.Remote) throws java.rmi.RemoteException, java.net.MalformedURLException;</code>                               | It binds the remote object to the new name.                                   |
| <code>public static java.lang.String[] list(java.lang.String) throws java.rmi.RemoteException, java.net.MalformedURLException;</code>                                    | It returns an array of the names of the remote objects bound in the registry. |

In this example, we are binding the remote object by the name sonoo.

1. **import** java.rmi.\*;
2. **import** java.rmi.registry.\*;
3. **public class** MyServer{
4. **public static void** main(String args[]){
5. **try**{
6. Adder stub=**new** AdderRemote();
7. Naming.rebind("rmi://localhost:5000/sonoo",stub);
8. }**catch**(Exception e){System.out.println(e);}
9. }
10. }

---

## 6) Create and run the client application

At the client we are getting the stub object by the lookup() method of the Naming class and invoking the method on this object. In this example, we are running the server and client applications, in the same machine so we are using localhost. If you want to access the remote object from another machine, change the localhost to the host name (or IP address) where the remote object is located.

```
1. import java.rmi.*;
2. public class MyClient{
3.     public static void main(String args[]){
4.         try{
5.             Adder stub=(Adder)Naming.lookup("rmi://localhost:5000/sonoo");
6.             System.out.println(stub.add(34,4));
7.         }catch(Exception e){}
8.     }
9. }
```

---

[download this example of rmi](#)

---

1. For running **this** rmi example,
- 2.
3. **1)** compile all the java files
- 4.
5. javac \*.java
- 6.
7. **2)**create stub and skeleton object by rmic tool
- 8.
9. rmic AdderRemote
- 10.
11. **3)**start rmi registry in one command prompt
- 12.
13. rmiregistry **5000**
- 14.
15. **4)**start the server in another command prompt

16.

17. `java MyServer`

18.

19. 5)start the client application in another command prompt

20.

21. `java MyClient`

Output of this RMI example

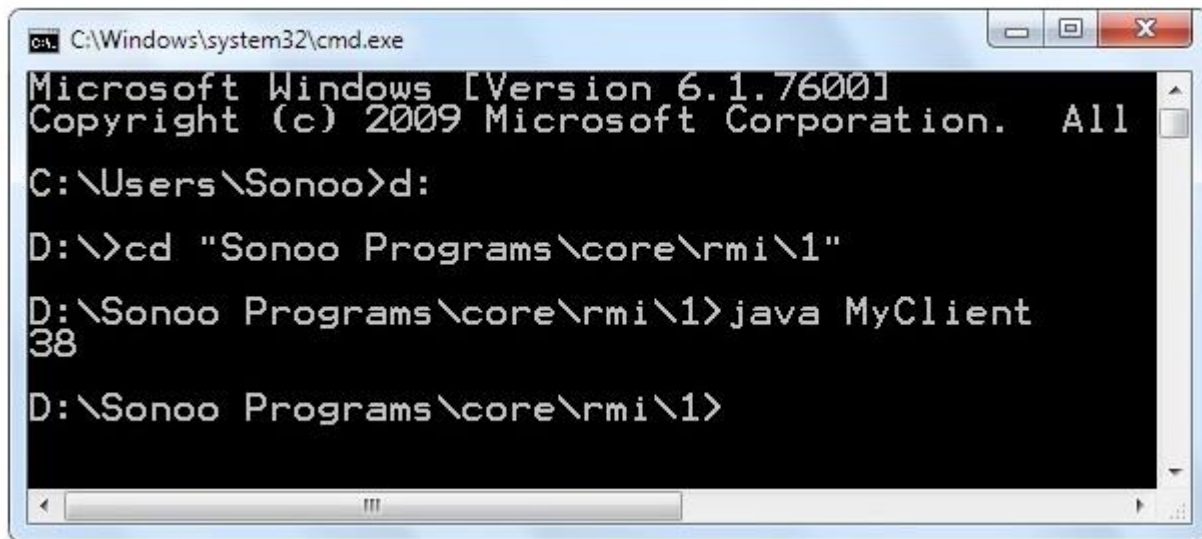


```
C:\Windows\system32\cmd.exe - rmiregistry 5000
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\Sonoo>d:
D:\>cd "Sonoo Programs\core\rmi\1"
D:\Sonoo Programs\core\rmi\1>javac *.java
D:\Sonoo Programs\core\rmi\1>rmic AdderRemote
D:\Sonoo Programs\core\rmi\1>rmiregistry 5000
```

```
C:\Windows\system32\cmd.exe - java MyServer
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\Sonoo>d:
D:\>cd "Sonoo Programs\core\rmi\1"
D:\Sonoo Programs\core\rmi\1>java MyServer
```



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\Sonoo>d:
D:\>cd "Sonoo Programs\core\rmi\1"
D:\Sonoo Programs\core\rmi\1>java MyClient
38
D:\Sonoo Programs\core\rmi\1>
```

---

## Meaningful example of RMI application with database

Consider a scenario, there are two applications running in different machines. Let's say MachineA and MachineB, machineA is located in United States and MachineB in India. MachineB want to get list of all the customers of MachineA application.

Let's develop the RMI application by following the steps.

### *1) Create the table*

First of all, we need to create the table in the database. Here, we are using Oracle10 database.

CUSTOMER400

TableDataIndexesModelConstraintsGrantsStatisticsUI DefaultsTriggersDependenciesSQL

QueryCount RowsInsert Row

| EDIT                                                                              | ACC_NO   | FIRSTNAME | LASTNAME | EMAIL                    | AMOUNT            |
|-----------------------------------------------------------------------------------|----------|-----------|----------|--------------------------|-------------------|
|  | 67539876 | James     | Franklin | franklin1james@gmail.com | 500000            |
|  | 67534876 | Ravi      | Kumar    | ravimalik@gmail.com      | 98000             |
|  | 67579872 | Vimal     | Jaiswal  | jaiswalvimal32@gmail.com | 9380000           |
|                                                                                   |          |           |          |                          | row(s) 1 - 3 of 3 |

Download

## 2) Create Customer class and Remote interface

File: Customer.java

1. **package** com.javatpoint;
2. **public class** Customer **implements** java.io.Serializable{
3.     **private int** acc\_no;
4.     **private** String firstname,lastname,email;
5.     **private float** amount;
6.     //getters and setters
7. }

Note: Customer class must be Serializable.

File: Bank.java

1. **package** com.javatpoint;
2. **import** java.rmi.\*;
3. **import** java.util.\*;
4. **interface** Bank **extends** Remote{
5.     **public** List<Customer> getCustomers()**throws** RemoteException;
6. }

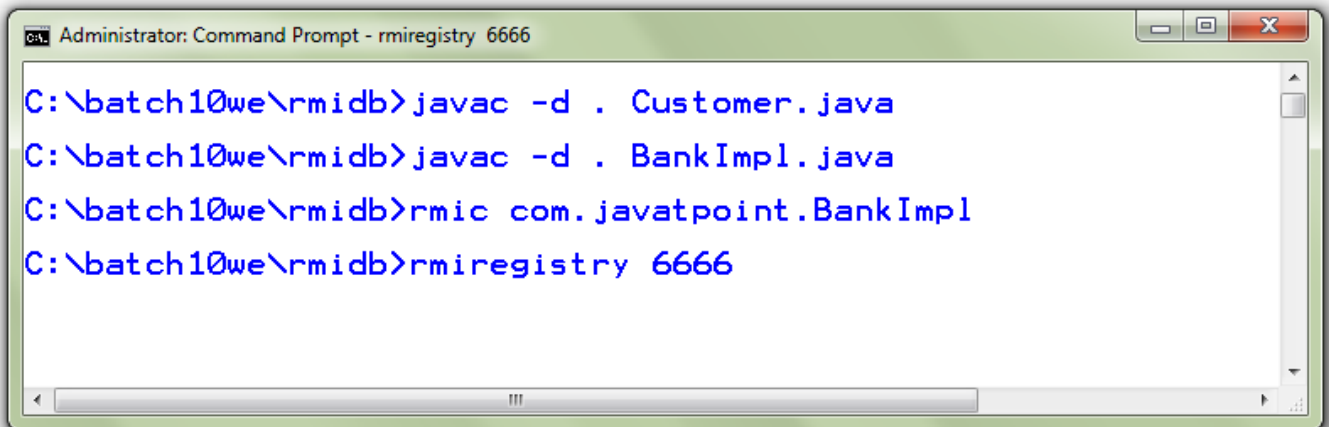
## 3) Create the class that provides the implementation of Remote interface

File: BankImpl.java

```
1. package com.javatpoint;
2. import java.rmi.*;
3. import java.rmi.server.*;
4. import java.sql.*;
5. import java.util.*;
6. class BankImpl extends UnicastRemoteObject implements Bank{
7. BankImpl()throws RemoteException{}
8.
9. public List<Customer> getCustomers(){
10. List<Customer> list=new ArrayList<Customer>();
11. try{
12. Class.forName("oracle.jdbc.driver.OracleDriver");
13. Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","
    oracle");
14. PreparedStatement ps=con.prepareStatement("select * from customer400");
15. ResultSet rs=ps.executeQuery();
16.
17. while(rs.next()){
18. Customer c=new Customer();
19. c.setAcc_no(rs.getInt(1));
20. c.setFirstname(rs.getString(2));
21. c.setLastname(rs.getString(3));
22. c.setEmail(rs.getString(4));
23. c.setAmount(rs.getFloat(5));
24. list.add(c);
25. }
26.
27. con.close();
28. }catch(Exception e){System.out.println(e);}
29. return list;
30. }//end of getCustomers()
31. }
```

---

4) Compile the class *rmic* tool and start the registry service by *rmiregistry* tool



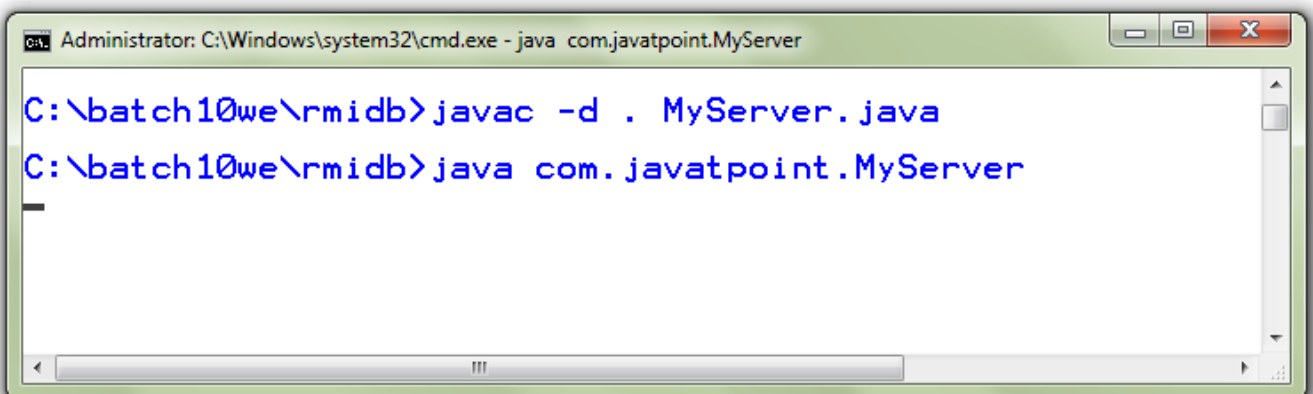
```
Administrator: Command Prompt - rmiregistry 6666

C:\batch10we\rmidb>javac -d . Customer.java
C:\batch10we\rmidb>javac -d . BankImpl.java
C:\batch10we\rmidb>rmic com.javatpoint.BankImpl
C:\batch10we\rmidb>rmiregistry 6666
```

5) Create and run the Server

File: *MyServer.java*

1. **package** com.javatpoint;
2. **import** java.rmi.\*;
3. **public class** MyServer{
4. **public static void** main(String args[])**throws** Exception{
5. Remote r=**new** BankImpl();
6. Naming.rebind("rmi://localhost:6666/javatpoint",r);
7. }}



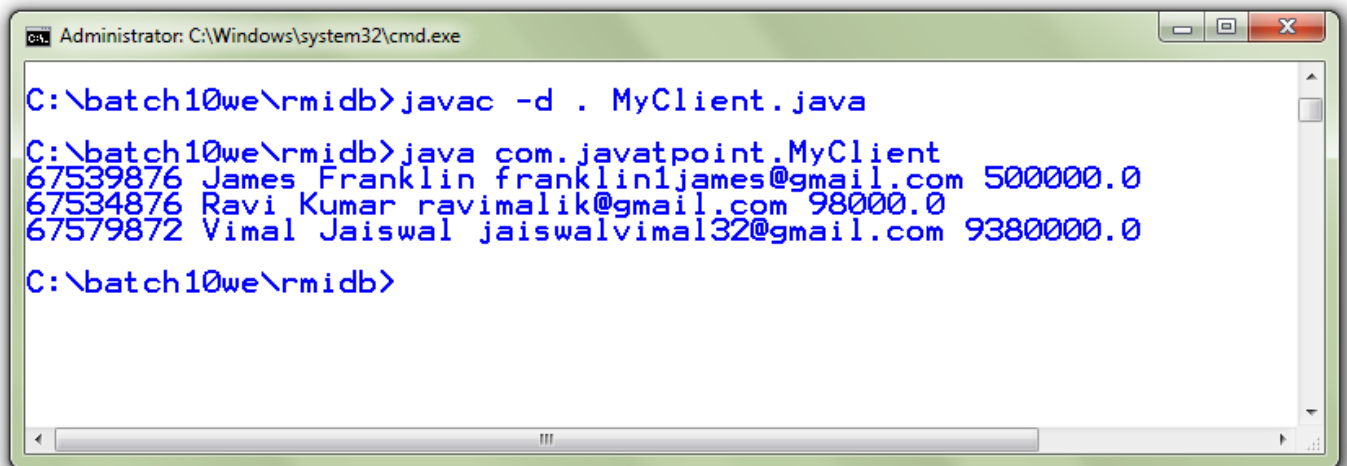
```
Administrator: C:\Windows\system32\cmd.exe - java com.javatpoint.MyServer

C:\batch10we\rmidb>javac -d . MyServer.java
C:\batch10we\rmidb>java com.javatpoint.MyServer
```

## 6) Create and run the Client

File: MyClient.java

1. **package** com.javatpoint;
2. **import** java.util.\*;
3. **import** java.rmi.\*;
4. **public class** MyClient{
5. **public static void** main(String args[]) **throws** Exception{
6. Bank b=(Bank)Naming.lookup("rmi://localhost:6666/javatpoint");
- 7.
8. List<Customer> list=b.getCustomers();
9. **for**(Customer c:list){
10. System.out.println(c.getAcc\_no()+" "+c.getFirstname()+" "+c.getLastname()
11. +" "+c.getEmail()+" "+c.getAmount());
12. }
- 13.
14. }}



```
Administrator: C:\Windows\system32\cmd.exe

C:\batch10we\rmidb>javac -d . MyClient.java

C:\batch10we\rmidb>java com.javatpoint.MyClient
67539876 James Franklin franklin1james@gmail.com 500000.0
67534876 Ravi Kumar ravimalik@gmail.com 98000.0
67579872 Vimal Jaishwal jaishwalvimal32@gmail.com 9380000.0

C:\batch10we\rmidb>
```

# Serialization and Deserialization in Java

**Serialization in Java** is a mechanism of *writing the state of an object into a byte-stream*. It is mainly used in Hibernate, RMI, JPA, EJB and JMS technologies.

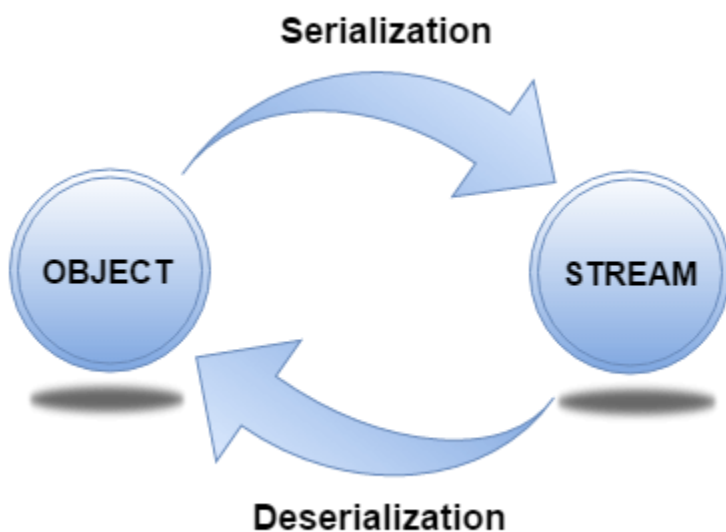
The reverse operation of serialization is called *deserialization* where byte-stream is converted into an object. The serialization and deserialization process is platform-independent, it means you can serialize an object on one platform and deserialize it on a different platform.

For serializing the object, we call the **writeObject()** method of *ObjectOutputStream* class, and for deserialization we call the **readObject()** method of *ObjectInputStream* class.

We must have to implement the *Serializable* interface for serializing the object.

## Advantages of Java Serialization

It is mainly used to travel object's state on the network (that is known as marshalling).



## java.io.Serializable interface

**Serializable** is a marker interface (has no data member and method). It is used to "mark" Java classes so that the objects of these classes may get a certain capability. The **Cloneable** and **Remote** are also marker interfaces.

The **Serializable** interface must be implemented by the class whose object needs to be persisted.

The String class and all the wrapper classes implement the *java.io.Serializable* interface by default.

Let's see the example given below:

### Student.java

```
1. import java.io.Serializable;
2. public class Student implements Serializable{
3.     int id;
4.     String name;
5.     public Student(int id, String name) {
6.         this.id = id;
7.         this.name = name;
8.     }
9. }
```

In the above example, **Student** class implements Serializable interface. Now its objects can be converted into stream. The main class implementation of is showed in the next code.

## ObjectOutputStream class

The ObjectOutputStream class is used to write primitive data types, and Java objects to an OutputStream. Only objects that support the java.io.Serializable interface can be written to streams.

### Constructor



|                                                                      |                                                                             |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 1) public ObjectOutputStream(OutputStream out) throws IOException {} | It creates an ObjectOutputStream that writes to the specified OutputStream. |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------|

### Important Methods

| Method                                                             | Description                                               |
|--------------------------------------------------------------------|-----------------------------------------------------------|
| 1) public final void writeObject(Object obj) throws IOException {} | It writes the specified object to the ObjectOutputStream. |
| 2) public void flush() throws IOException {}                       | It flushes the current output stream.                     |
| 3) public void close() throws IOException {}                       | It closes the current output stream.                      |

## ObjectInputStream class

An ObjectInputStream deserializes objects and primitive data written using an ObjectOutputStream.

### Constructor

|                                                                   |                                                                            |
|-------------------------------------------------------------------|----------------------------------------------------------------------------|
| 1) public ObjectInputStream(InputStream in) throws IOException {} | It creates an ObjectInputStream that reads from the specified InputStream. |
|-------------------------------------------------------------------|----------------------------------------------------------------------------|

### Important Methods

| Method                                                                            | Description                               |
|-----------------------------------------------------------------------------------|-------------------------------------------|
| 1) public final Object readObject() throws IOException, ClassNotFoundException {} | It reads an object from the input stream. |

```
2) public void close() throws IOException {}
```

It closes ObjectInputStream.

## Example of Java Serialization

In this example, we are going to serialize the object of **Student** class from above code. The writeObject() method of ObjectOutputStream class provides the functionality to serialize the object. We are saving the state of the object in the file named f.txt.

### Persist.java

```
1. import java.io.*;
2. class Persist{
3.     public static void main(String args[]){
4.         try{
5.             //Creating the object
6.             Student s1 =new Student(211,"ravi");
7.             //Creating stream and writing the object
8.             FileOutputStream fout=new FileOutputStream("f.txt");
9.             ObjectOutputStream out=new ObjectOutputStream(fout);
10.            out.writeObject(s1);
11.            out.flush();
12.            //closing the stream
13.            out.close();
14.            System.out.println("success");
15.        }catch(Exception e){System.out.println(e);}
16.    }
17. }
```

### Output:

```
success
```

## Example of Java Deserialization

Deserialization is the process of reconstructing the object from the serialized state. It is the reverse operation of serialization. Let's see an example where we are reading the data from a deserialized object.

Deserialization is the process of reconstructing the object from the serialized state. It is the reverse operation of serialization. Let's see an example where we are reading the data from a deserialized object.

### Depersist.java

```
1. import java.io.*;
2. class Depersist{
3.     public static void main(String args[]){
4.         try{
5.             //Creating stream to read the object
6.             ObjectInputStream in=new ObjectInputStream(new FileInputStream("f.txt"));
7.             Student s=(Student)in.readObject();
8.             //printing the data of the serialized object
9.             System.out.println(s.id+ " "+s.name);
10.            //closing the stream
11.            in.close();
12.        }catch(Exception e){System.out.println(e);}
13.    }
14. }
```

### Output:

```
211 ravi
```

---

## Java Serialization with Inheritance (IS-A Relationship)

If a class implements **Serializable interface** then all its sub classes will also be serializable. Let's see the example given below:

### SerializeSA.java

```
1. import java.io.Serializable;
2. class Person implements Serializable{
3.     int id;
4.     String name;
```

```

5. Person(int id, String name) {
6.     this.id = id;
7.     this.name = name;
8. }
9. }
10. class Student extends Person{
11.     String course;
12.     int fee;
13.     public Student(int id, String name, String course, int fee) {
14.         super(id,name);
15.         this.course=course;
16.         this.fee=fee;
17.     }
18. }
19. public class SerialzeISA
20. {
21.     public static void main(String args[])
22.     {
23.         try{
24.             //Creating the object
25.             Student s1 =new Student(211,"ravi","Engineering",50000);
26.             //Creating stream and writing the object
27.             FileOutputStream fout=new FileOutputStream("f.txt");
28.             ObjectOutputStream out=new ObjectOutputStream(fout);
29.             out.writeObject(s1);
30.             out.flush();
31.             //closing the stream
32.             out.close();
33.             System.out.println("success");
34.         }catch(Exception e){System.out.println(e);}
35.     }try{
36.         //Creating stream to read the object
37.         ObjectInputStream in=new ObjectInputStream(new FileInputStream("f.txt"));
38.         Student s=(Student)in.readObject();

```

```

39. //printing the data of the serialized object
40. System.out.println(s.id+" "+s.name+" "+s.course+" "+s.fee);
41. //closing the stream
42. in.close();
43. }catch(Exception e){System.out.println(e);}
44. }
45. }

```

### Output:

```

success
211 ravi Engineering 50000

```

The `SerializeSA` class has serialized the `Student` class object that extends the `Person` class which is `Serializable`. Parent class properties are inherited to subclasses so if parent class is `Serializable`, subclass would also be.

## Java Serialization with Aggregation (HAS-A Relationship)

If a class has a reference to another class, all the references must be `Serializable` otherwise serialization process will not be performed. In such case, *`NotSerializableException`* is thrown at runtime.

### Address.java

```

1. class Address{
2.     String addressLine,city,state;
3.     public Address(String addressLine, String city, String state) {
4.         this.addressLine=addressLine;
5.         this.city=city;
6.         this.state=state;
7.     }
8. }

```

### Student.java

```

1. import java.io.Serializable;

```

```
2. public class Student implements Serializable{
3.     int id;
4.     String name;
5.     Address address;//HAS-A
6.     public Student(int id, String name) {
7.         this.id = id;
8.         this.name = name;
9.     }
10. }
```

Since Address is not Serializable, you cannot serialize the instance of the Student class.

*Note: All the objects within an object must be Serializable.*

## Java Serialization with the static data member

If there is any static data member in a class, it will not be serialized because static is the part of class not object.

### Employee.java

```
1. class Employee implements Serializable{
2.     int id;
3.     String name;
4.     static String company="SSS IT Pvt Ltd";//it won't be serialized
5.     public Student(int id, String name) {
6.         this.id = id;
7.         this.name = name;
8.     }
9. }
```

## Java Serialization with array or collection

Rule: In case of array or collection, all the objects of array or collection must be serializable. If any object is not serializable, serialization will be failed.

## Externalizable in java

The Externalizable interface provides the facility of writing the state of an object into a byte stream in compress format. It is not a marker interface.

The Externalizable interface provides two methods:

- **public void writeExternal(ObjectOutput out) throws IOException**
- **public void readExternal(ObjectInput in) throws IOException**

## Java Transient Keyword

If you don't want to serialize any data member of a class, you can mark it as transient.

### Employee.java

1. **class** Employee **implements** Serializable{
2.   **transient int** id;
3.   String name;
4.   **public** Student(**int** id, String name) {
5.     **this**.id = id;
6.     **this**.name = name;
7.   }
8. }

Now, id will not be serialized, so when you deserialize the object after serialization, you will not get the value of id. It will return default value always. In such case, it will return 0 because the data type of id is an integer.

Visit next page for more details.

## SerialVersionUID

The serialization process at runtime associates an id with each Serializable class which is known as serialVersionUID. It is used to verify the sender and receiver of the serialized object. The sender and receiver must be the same. To verify it, serialVersionUID is used. The sender and receiver must have the same serialVersionUID, otherwise, **InvalidClassException** will be thrown when you deserialize the object. We

can also declare our own serialVersionUID in the Serializable class. To do so, you need to create a field serialVersionUID and assign a value to it. It must be of the long type with static and final. It is suggested to explicitly declare the serialVersionUID field in the class and have it private also. For example:

1. **private static final long** serialVersionUID=1L;

Now, the Serializable class will look like this:

### **Employee.java**

1. **import** java.io.Serializable;
2. **class** Employee **implements** Serializable{
3. **private static final long** serialVersionUID=1L;
4. **int** id;
5. String name;
6. **public** Student(**int** id, String name) {
7. **this**.id = id;
8. **this**.name = name;
9. }
10. }